

北京邮电大学设计实践报告

系统设计名称	课程学习助手 Agent 平台	学 院	网络空间 安全学院	指导教师	昌 硕
班 级	班内序号	学 号	学生姓名	成绩	
2024211803		2024211509	吴锦澄		
系统设计内容	完成用户认证、课程与资料管理、多格式资料解析、基于资料的问答与引用、知识点与知识图谱、学习计划与待办、测验与薄弱点、模型配置、Agent 审计及自动化测试发布。				
学生设计报告 (附页)					
设计成绩评定	指导教师签名： 年 月 日				

注：评语要体现每个学生的工作情况，可以加页。

- 一、课程设计目的
- 二、需求分析
- 三、详细设计
- 四、难点与亮点
- 五、设计成果
- 六、设计心得
- 七、团队分工说明
- 八、AI 伦理声明

一、课程设计目的

本课程设计面向大学生多门课程学习过程中资料分散、知识查找效率低、学习计划难以持续执行以及复习任务容易遗漏等问题，设计并实现“课程学习助手 Agent 平台”。平台以课程为数据边界，将课件、教材笔记、作业要求、实验指导书、知识点、问答记录、计划和待办统一组织，使学生能够在一个系统中完成资料管理、理解、复习和任务跟踪。

本项目的首要目的不是简单地增加一个大模型聊天页面，而是将大模型能力约束在课程资料和可验证证据之上。系统在回答问题时先检索课程资料，再生成带有来源片段、页码和支撑理由的回答；当证据不足时明确拒绝给出确定性课程结论。通过这一机制，平台降低了通用大模型在专业课程问答中出现“回答流畅但缺乏依据”的风险。

第二个目的是按照软件工程方法完成一套可运行、可维护、可测试和可发布的完整系统。项目覆盖需求分析、总体设计、数据库设计、前后端实现、资料解析、Agent 服务、测试、迁移、持续集成和版本发布，并通过 GitHub Actions 建立后端、前端、数据库迁移、端到端测试和功能闭环验收门禁。

第三个目的是形成从课程资料到学习反馈的闭环。资料解析后可生成知识点和知识图谱；学生可据此生成学习计划和每日待办；完成学习后通过测验记录错题和薄弱点，后续计划再根据执行结果进行调整。系统由此从“资料存储工具”扩展为“学习过程管理工具”。

1.1 具体实践目标

掌握 Vue 3、TypeScript、FastAPI、SQLAlchemy 与 SQLite 的前后端分离开发方法。

实现 TXT、Markdown、PDF、DOCX、PPTX 等多种课程资料的统一上传、解析和管理。理解并实现基于全文检索的检索增强生成流程，以及引用校验和证据不足处理。

完成知识点、知识图谱、学习计划、待办、测验、错题和薄弱点等业务模块的关联设计。

建立 JWT 鉴权、密码哈希、API Key 加密、数据隔离和运行审计等安全机制。

通过 `pytest`、`Vitest`、`Playwright` 和 `GitHub Actions` 对系统进行自动化验证并发布稳定版本。

1.2 设计范围与实现边界

系统默认以 `SQLite` 作为数据库，以 `SQLite FTS5` 和 `BM25` 作为主要检索机制，在 `FTS5` 不可用或无结果时退化为关键词匹配。当前代码保留了向量检索接口，但稳定版本没有启用向量召回，因此报告不将向量数据库作为已实现能力。真实模型通过 `OpenAI-compatible Chat Completions` 接口接入；无密钥环境可以使用确定性的 `Mock` 模式演示流程。

系统重点服务于个人或小规模本地课程学习场景，提供 `Windows` 一键启动脚本。对于大规模多租户部署、跨数据库迁移、分布式向量检索和高并发异步任务队列等能力，本版本仅保留扩展空间，不作为课程设计完成范围。

二、需求分析

2.1 项目背景与用户需求

大学课程资料通常散落在教学平台、聊天群、个人文件夹和笔记软件中。同一门课程可能包含多份课件、教材章节、实验指导和作业说明，学生在复习时需要频繁切换文件并手工定位内容。与此同时，多门课程的考试和作业截止时间相互交叉，仅依靠临时记忆容易造成任务遗漏。

目标用户为高校学生。用户希望平台能够按照课程组织资料，快速检索某一知识点所在页，基于资料回答问题，自动整理重点内容，并把“复习完一门课”这类模糊目标拆成可执行的阶段任务和每日待办。用户还需要查看任务完成历史、测验成绩和薄弱知识点，从而知道下一步应复习什么。

2.2 功能需求

表 2-1 系统功能需求

功能模块	需求说明
用户认证	注册、登录、JWT 会话校验、退出；不同用户业务数据相互隔离。
课程管理	创建、编辑、删除和查询课程，记录课程名称、教师、学期、简介和主题色。
资料管理	上传 TXT、MD、PDF、DOCX、PPTX；查看状态、版本和解析结果；支持重新解析。
资料学习	按页或结构化文本阅读，展示目录、阅读进度、原页预览和文档内容。
课程问答	在指定课程范围内检索资料并进行多轮问答，返回引用和证据状态。
知识整理	从课程资料提取知识点、重要度、考查方式和复习建议，并形成图谱关系。
学习计划	根据目标、截止时间和每日可用时间生成阶段任务；支持多课程综合排程。
待办管理	按日期展示每日任务，支持开始、暂停、完成、撤销和实际用时记录。
测验与薄弱点	生成选择题、多选题、判断题和简答题，记录答案、解析、错题与掌握度。
模型配置	用户维护 OpenAI-compatible 配置，测试连接、启用模型并脱敏展示密钥。

Agent 审计	记录输入、输出、步骤、耗时、请求模型、实际模型、回退和证据状态。
日志与运维	健康检查、错误日志、数据库迁移、一键启动、自动化测试和版本发布。

2.3 非功能需求与约束

表 2-2 非功能需求

类别	要求
安全性	密码采用 bcrypt 哈希；JWT 用于访问控制；用户级 API Key 经 Fernet 加密，前端只接收脱敏值。
可靠性	解析、检索和大模型调用均保留状态及错误信息；证据不足时不伪造引用；迁移脚本支持 dry-run。
可维护性	前后端分层，Pydantic Schema、服务层、ORM 模型和 API 端点职责分离；统一异常处理。
可测试性	后端单元/集成测试、前端类型检查与单元测试、Playwright 端到端测试、功能闭环验收。
易用性	统一左侧导航；课程详情集中展示资料、问答、知识点、计划、测验和图谱入口。
可追溯性	资料版本、切块来源、引用、测验证据、Agent 运行和步骤均保存关联记录。
兼容性	Python 3.10+、Node.js 20.19+；Windows 提供启动、停止与桌面快捷方式脚本。
性能策略	CRUD 请求同步完成；资料解析可记录任务状态；检索采用 SQLite FTS5/BM25，避免每次遍历全部文本。

2.4 典型使用场景

场景一：建立课程空间：用户登录后创建“计算机网络”课程，填写教师和学期，上传课件。系统解析文件并生成页、结构块和检索切块。

场景二：基于资料问答：用户在课程问答中提问。系统检索相关片段并调用模型生成回答；引用验证通过时显示资料、页码和原文，证据不足时返回安全提示。

场景三：制定复习计划：用户输入截止日期和每日可用时间，系统将目标拆分为知识学习、复习和测验任务，写入每日待办。

场景四：测验反馈：用户完成测验后，系统计算得分，记录错题并更新薄弱点，为后续复习提供依据。

场景五：审计模型运行：用户在 Agent 审计页查看某次问答的请求模型、实际模型、耗时、输入输出和步骤，判断是否发生回退或证据不足。

三、详细设计

3.1 总体架构



图 3-1 系统总体架构

系统采用前后端分离结构。浏览器端负责交互和状态管理；FastAPI 提供统一 REST API；业务服务负责课程、资料、计划、测验等领域逻辑；Agent 与检索层组合全文检索和模型调用；SQLite、上传目录与解析目录持久化业务数据和文档产物。

前端使用 Vue Router 组织登录、仪表盘、课程、资料、课程问答、文档学习、知识点大纲、计划、测验、待办、知识图谱、日志中心、Agent 审计和个人中心等页面。路由守卫在进入受保护页面前调用后端校验当前用户，避免只相信浏览器本地保存的旧 Token。

后端按照 API、Schema、Service、Model、Retrieval 和 Agent 分层。API 负责鉴权与参数边界；Service 编排业务事务；SQLAlchemy ORM 管理关系数据；Retrieval 负责解析、清洗、切块与检索；Agent 负责提示词版本、模型调用、结构化输出和运行记录。

3.2 技术栈与工程结构

表 3-1 技术栈

层次	主要技术	作用
后端框架	FastAPI 、 Pydantic 2 、 SQLAlchemy 2.x	提供异步友好的 REST API、参数校验和 ORM。
数据库与检索	SQLite、FTS5、BM25	保存业务数据并完成课程内全文检索。
文档解析	PyMuPDF 、 pypdf 、 python-docx、python-pptx	解析 PDF、DOCX、PPTX，并保留页码和结构信息。
前端	Vue 3、TypeScript、Vite、Element Plus	实现组件化页面、类型检查和生产构建。
状态与路由	Pinia、Vue Router	管理认证状态、页面导航和路由鉴权。
图谱展示	vis-network	展示知识节点、关系和交互筛选。
认证安全	JWT 、 python-jose 、 passlib/bcrypt、Fernet	完成登录会话、密码哈希和 API Key 加密。
测试与发布	pytest、Vitest、Playwright、GitHub Actions	覆盖后端、前端、端到端、迁移和验收。

工程根目录下分为 backend 、 frontend 、 scripts 、 docs/engineering 和 .github/workflows。后端 app 目录进一步划分 agents、api/v1/endpoints、core、db、models、retrieval、schemas、services 和 tests；前端 src 划分 api、components、layouts、router、stores 和 views。运行数据库、用户上传资料、解析结果、日志、测试报告和凭据均通过 .gitignore 排除，不进入源代码版本。

3.3 数据库设计

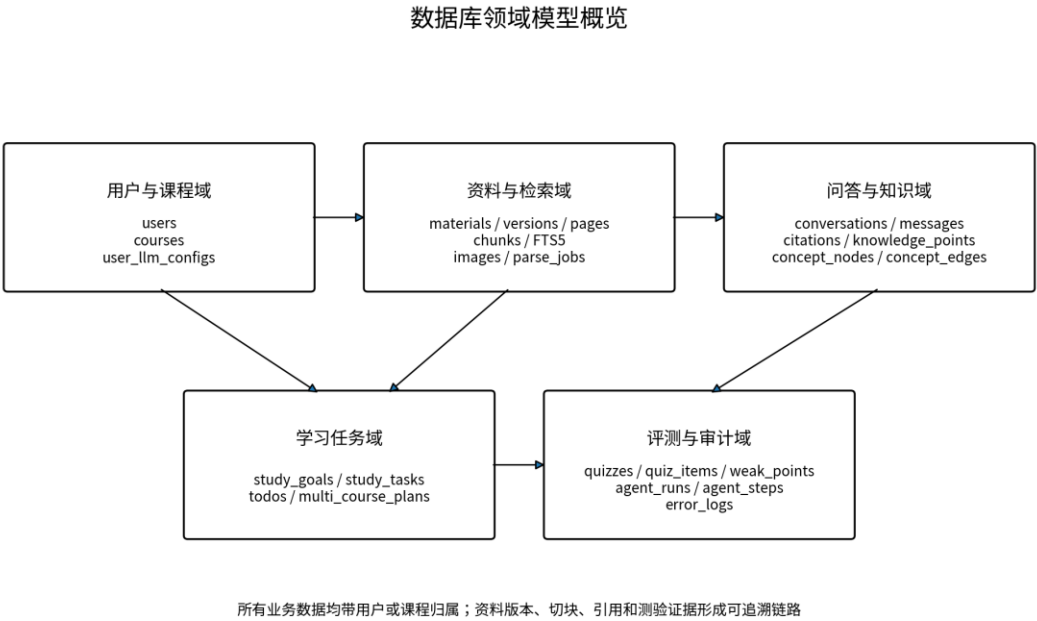


图 3-2 数据库领域模型概览

数据库设计以用户和课程为主线。课程关联资料、知识点、对话、计划、待办和测验；资料通过 `material_versions` 支持版本化，通过 `material_pages`、`material_chunks` 和 `material_images` 保存解析结构；回答引用 `citations` 指向具体切块并记录页码、引用原文和验证状态。

表 3-2 核心数据库实体

核心实体	设计说明
users / courses	用户与课程基础信息；courses.user_id 实现数据隔离。
materials / material_versions	记录文件元数据、解析状态、活动版本和失败原因。
material_pages / material_chunks	保存页级结构、清洗文本、切块来源、质量分数和可索引状态。
conversations / messages / citations	保存多轮问答、结构化回答以及引用验证结果。
knowledge_points / concept_nodes	保存知识点、概念节点、关系类型、置信度和审核状态。

concept_edges	
study_goals / study_tasks / todos	保存目标拆解、阶段任务、日程、执行和完成记录。
quizzes / quiz_items / weak_points	保存测验、题目证据、用户作答、错题和掌握度。
agent_runs / agent_steps	保存 Agent 运行元数据、输入输出摘要、步骤和回退链路。

本地验收备份数据库中已经积累了 68 门课程、94 份资料、3873 个资料切块、232 个知识点、202 条待办、9 份测验和 134 次 Agent 运行记录。该规模能够验证主要表之间的关联、分页查询和资料检索流程。

3.4 资料解析与检索问答

3.4.1 多格式资料解析

上传接口首先校验用户、课程、文件扩展名和大小，将原文件保存到用户/课程/资料分层目录，并创建资料版本与解析任务。不同解析器把 TXT、Markdown、PDF、DOCX 和 PPTX 转换为统一的页、块和文本结构。PDF/PPTX 额外保留页码、阅读顺序、标题、列表、表格及图片锚点，便于原页预览和引用定位。

切块阶段对空白、页眉页脚、异常字符和低质量文本进行清洗，记录 raw_text、cleaner_version、noise_score、source_block_ids 和 source_fragments 等追溯字段。资料重新解析时生成新版本并切换 active_version_id，旧版本不直接覆盖，便于迁移和审计。

3.4.2 检索与回答生成

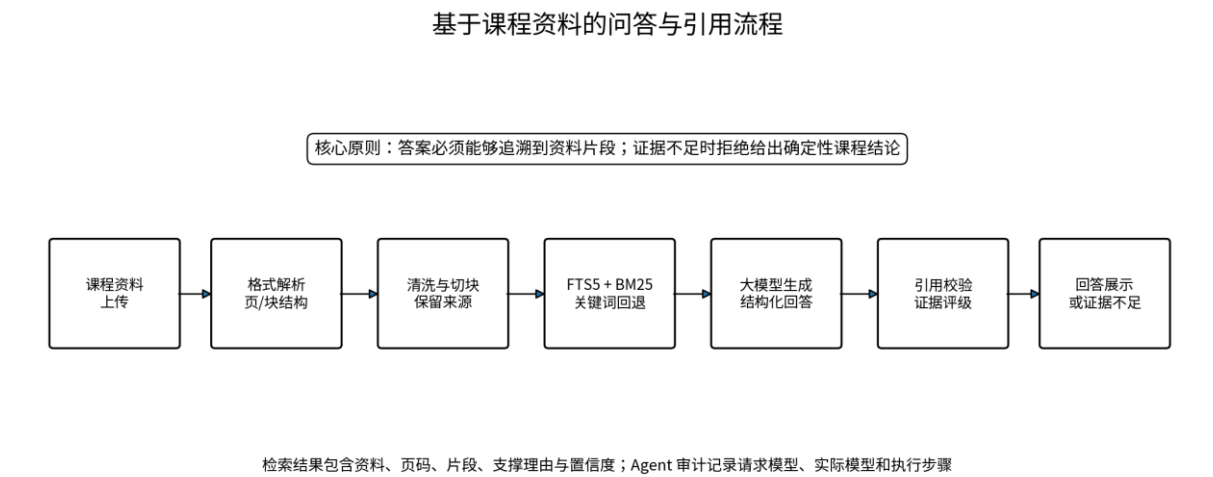


图 3-3 基于课程资料的问答流程

系统在指定课程范围内对用户问题进行别名扩展和查询改写，优先调用 SQLite FTS5 的 BM25 排序；当 FTS5 不可用或没有结果时，使用 LIKE 关键词回退，并提高标题和资料名命中的权重。候选片段在进入模型前保留 `chunk_id`、`material_id`、`page_no` 和原文。

模型输出采用结构化字段，包含回答、引用候选、支撑理由和置信度。引用验证器检查引用原文是否存在于对应切块、回答中的关键主张是否得到片段支撑。如果验证结果不足，系统把运行状态标记为“证据不足”，不将模型补全内容伪装为课程结论。

候选片段 = FTS5_BM25(课程 ID, 改写后的问题)

如果候选片段为空：

候选片段 = 关键词回退检索(标题权重 2，资料名权重 2，正文权重 1)

结构化回答 = 调用模型(问题，候选片段，提示词版本)

验证结果 = 校验引用原文与主张支撑关系

如果验证结果不足：

返回“证据不足”并记录审计

否则：

返回回答、资料名、页码、原文和支撑理由

3.5 知识整理、计划与评测

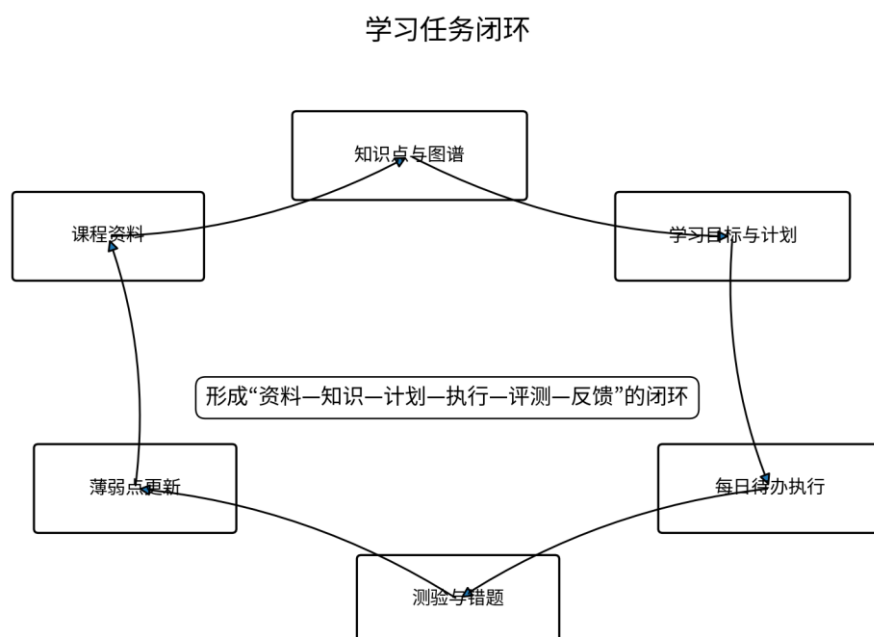


图 3-4 学习任务闭环

3.5.1 知识点与知识图谱

知识点生成服务以课程资料切块为输入，输出标题、摘要、重要度、考查方式、复习建议和来源切块集合。知识点使用规范化标题和 `stable_key` 去重，支持重新生成和代际管理。知识图谱将知识点映射为概念节点，边记录相似、对比、前置、迁移应用、同名异义、易混和上下位等关系，并保存关系理由、证据和确认状态。

3.5.2 学习计划和待办

用户输入学习目标、截止日期和每日可用时间后，Planner 把目标拆分为阶段任务。每个任务包含课程、任务类型、预计时长、优先级、完成标准和可跳转的目标资料或测验。排程服务按截止时间、优先级、预计时长和每日容量分配日期，生成待办；多课程计划同时计算各课程截止风险和每日负载。

按截止日期和优先级排序阶段任务

for 每个任务：

将任务放入截止日期前仍有剩余容量的最早日期

若单日容量不足，则拆分或标记未排程原因

生成待办并记录计划代次
任务完成后写入实际用时、完成时间和验证结果

3.5.3 测验、错题与薄弱点

测验生成支持选择题、多选题、判断题和简答题，可限定题数、难度、知识点和及格分。题目保存来源证据快照和评分规则；提交后自动判定客观题，简答题依据 rubric 处理。错误答案更新 weak_points 的 wrong_count、mastery_score 和最近练习时间，连续答对后可逐步降低薄弱程度。

3.6 安全、审计与异常处理

身份认证采用用户名和密码登录，密码只保存 bcrypt 哈希。受保护 API 从 JWT 中解析用户身份，并在课程、资料、对话和计划查询中继续校验 user_id，避免越权访问。生产环境必须替换 JWT 和 Fernet 默认密钥，并限制 CORS 来源。

用户级模型配置保存 provider、Base URL、model、temperature、max_tokens 和 timeout。API Key 使用 Fernet 加密后写入数据库，查询接口只返回脱敏结果。连接测试访问兼容的 /chat/completions 接口，同时验证网络、鉴权、模型名和响应结构。

每次 Agent 调用创建 agent_runs 记录，保存 run_type、status、prompt_version、请求 provider/model、实际 provider/model、config_id、fallback_used、fallback_reason、evidence_status、输入输出摘要和耗时；agent_steps 进一步记录 query_rewrite、retrieval、generation、citation_verification 等步骤。该设计使真实模型调用、Mock 演示和回退结果能够明确区分。

3.7 测试与持续集成

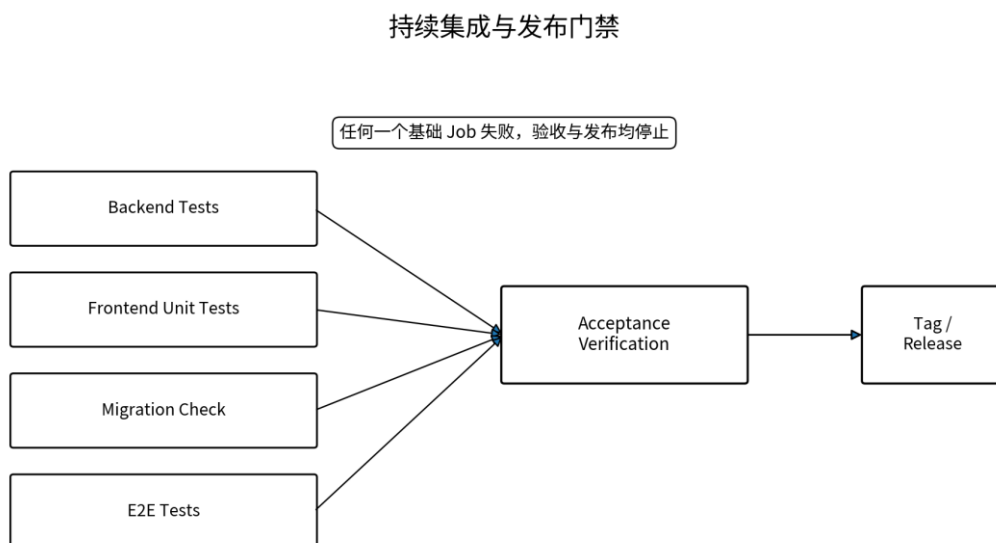


图 3-5 持续集成与发布门禁

CI 在 main 分支推送、Pull Request 和手工触发时运行。Backend Tests 安装 Python 依赖并执行 pytest; Frontend Unit Tests 执行 TypeScript 类型检查、Vitest 和生产构建; Migration Check 初始化隔离数据库并执行 dry-run 和 smoke test; E2E Tests 在独立 SQLite 与存储目录中运行 Playwright Chromium; Acceptance Verification 依赖前四项全部成功，并验证功能闭环结果。

发布工作流只有在 main 的 CI 完成后才能创建 v1.0.0 Tag 和 GitHub Release。测试日志、迁移结果、Playwright 报告和验收产物作为 Actions Artifact 保存，而数据库、缓存、虚拟环境、日志和本地凭据不进入 Git 索引。

四、难点与亮点

4.1 多格式文档的结构化解析

课程资料并不是纯文本集合。PDF 可能包含双栏排版、表格、图片和页眉页脚；PPTX 由多个文本框和形状组成；DOCX 还包含标题层级和表格。若只进行简单文本抽取，页面顺序、页码和引用位置容易丢失。系统将不同格式统一映射为“资料—版本—页—块—一切块”结构，并保留来源块、页码、清洗版本和内容哈希，使文档阅读、检索和引用可以共享同一套数据。

4.2 可靠引用与证据不足机制

本项目最重要的亮点是将回答和证据绑定。系统不仅展示“引用了哪份资料”，还保存引用原文、页码、主张、支撑状态、验证理由和验证器版本。模型没有找到可验证证据时，页面明确提示用户查看检索过程或更换问法，而不是继续生成看似合理的答案。该机制把大模型从不可控的自由生成组件转变为受课程资料约束的辅助工具。

4.3 用户级真实模型配置与可审计回退

为了兼顾无密钥演示和真实模型效果，系统实现“用户配置—系统配置—Mock”的调用顺序。用户可在个人中心配置任意 OpenAI-compatible 模型，密钥加密存储且只脱敏显示；实际调用失败时，系统记录回退原因和实际模型。Agent 审计页面可对照请求 Provider 与实际 Provider，避免把 Mock 或回退结果误认为真实模型结果。

4.4 从资料到评测反馈的学习闭环

知识点、计划、待办、测验和薄弱点并非孤立页面。知识点保存资料来源；计划任务可跳转到对应资料或测验；待办完成状态回写计划；测验错题更新薄弱点；薄弱点又可作为后续复习重点。通过跨模块关联，系统能够记录学习过程而不是只保存最终答案。

4.5 工程化验收与可发布性

课程设计通常容易停留在“本机运行”的状态。本项目额外建立数据库兼容迁移、Git 索引卫生检查、端到端隔离环境、功能闭环验收和发布工作流。GitHub Actions

中五个主要 Job 全部通过后才允许发布稳定版本，Release 页面同时给出功能边界和验收条件，提高了成果的可复现性。

表 4-1 难点与解决方案

主要难点	解决方法
文档结构差异大	统一页/块/切块模型，保留页码、来源块和版本。
大模型可能产生幻觉	检索优先、引用验证、证据评级和证据不足拒答。
真实模型配置涉及密钥	Fernet 加密、接口脱敏、连接测试和用户隔离。
计划任务容易失真	保存预计时长、完成标准、计划代次、执行状态和实际用时。
旧数据库结构变化	迁移 dry-run、语义化外键比较、smoke test 和回滚验证。
功能多且回归风险高	pytest、Vitest、Playwright、Acceptance 和发布门禁。

五、设计成果

系统已形成可运行的前后端应用，并完成水墨沉浸式前端改造。本节结合新版真实运行截图说明主要界面和验收结果。截图中的测试账号仅用于本地演示；模型 API Key 均以脱敏形式展示。

5.1 登录与仪表盘

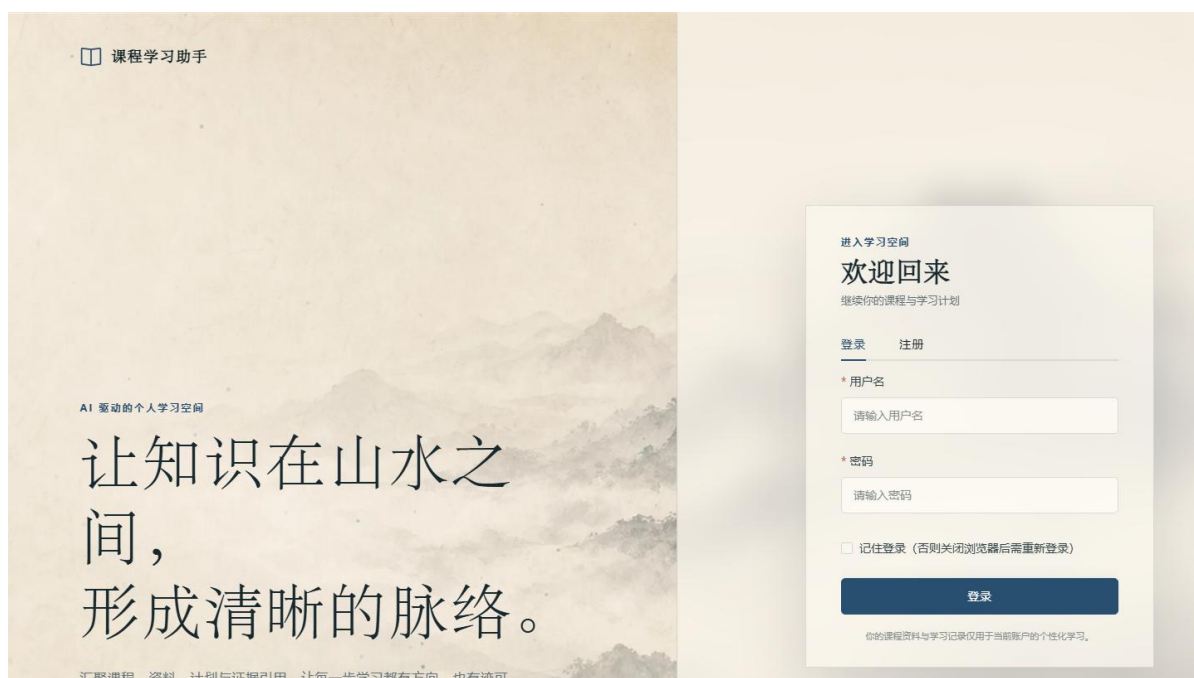


图 5-1 用户登录与注册页面

登录页面以留白、水墨山水和低速粒子构成沉浸式入口，同时保留登录、注册和记住登录状态。后端对密码进行哈希验证，登录成功后签发 JWT；路由守卫在进入业务页面前再次校验当前用户。



图 5-2 系统仪表盘

仪表盘以中央山水学习画布和深色学习轨迹侧栏组织信息，集中展示当前课程、下一步行动、AI 学习建议、今日计划和近期运行记录，为用户提供清晰的学习主线。

5.2 课程创建与课程空间



图 5-3 新建课程界面

新版纸感弹窗保持课程名称、授课教师、学期、课程简介和主题色等完整字段。课

程信息由前端表单校验后写入后端，并自动归属于当前用户。

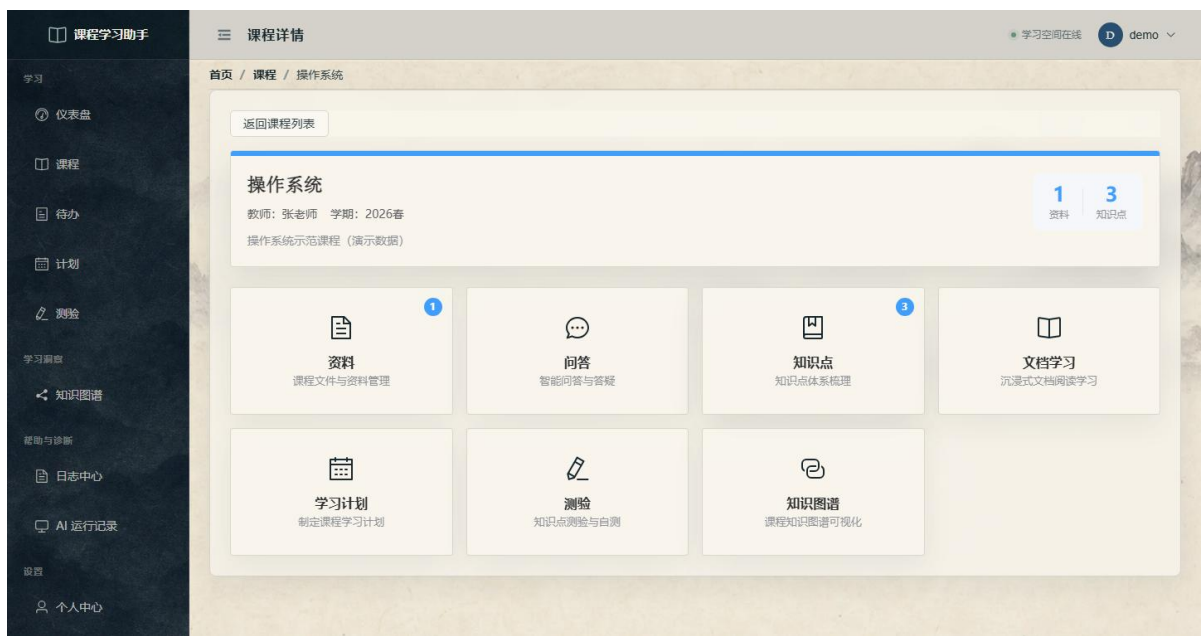


图 5-4 课程详情与功能入口

课程空间延续宣纸与墨色视觉语言，以课程为中心汇总资料、问答、知识点、文档学习、学习计划、测验和知识图谱入口；统计值来自当前演示数据。

5.3 课程资料管理

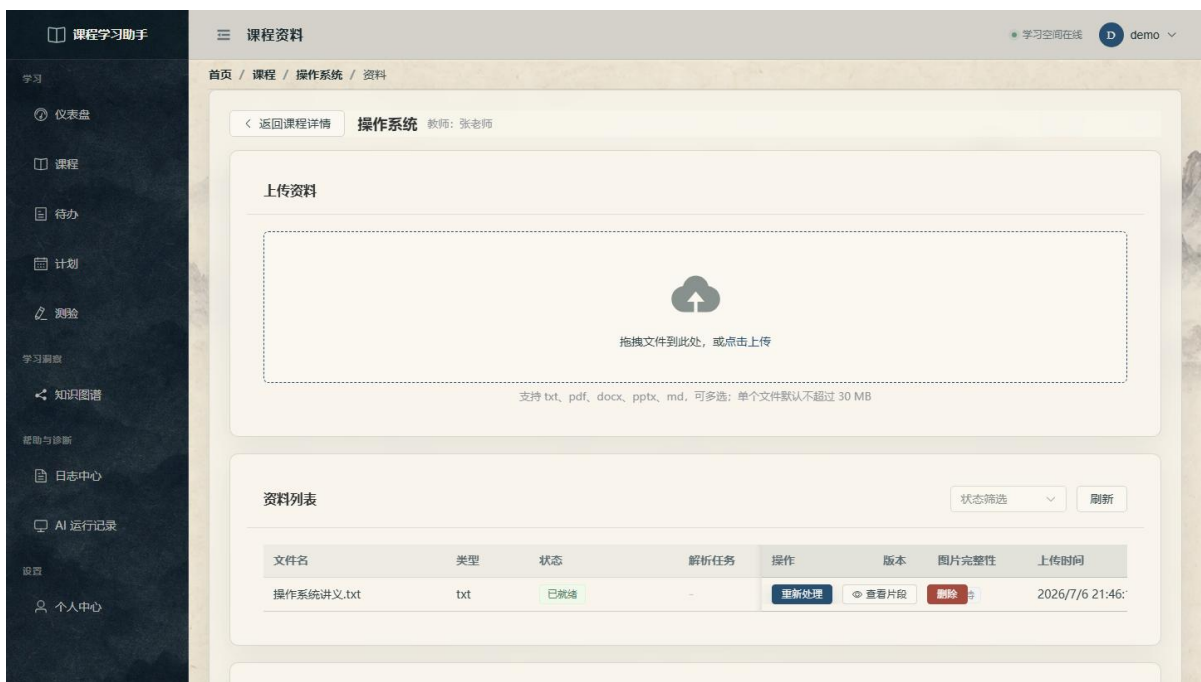


图 5-5 资料上传与解析状态

资料页面在统一的纸感信息层中支持拖拽或点击上传，限制文件类型和大小，并显示资料类型、状态、版本、图片完整性、上传时间和操作。解析完成的资料可查看片段或重新处理。

5.4 知识点整理与知识图谱



图 5-6 知识点大纲

知识点大纲在新版界面中展示标题、摘要、重要度、考查方式、建议任务和来源片段，并允许重新生成；数量与状态均以当前课程数据为准。

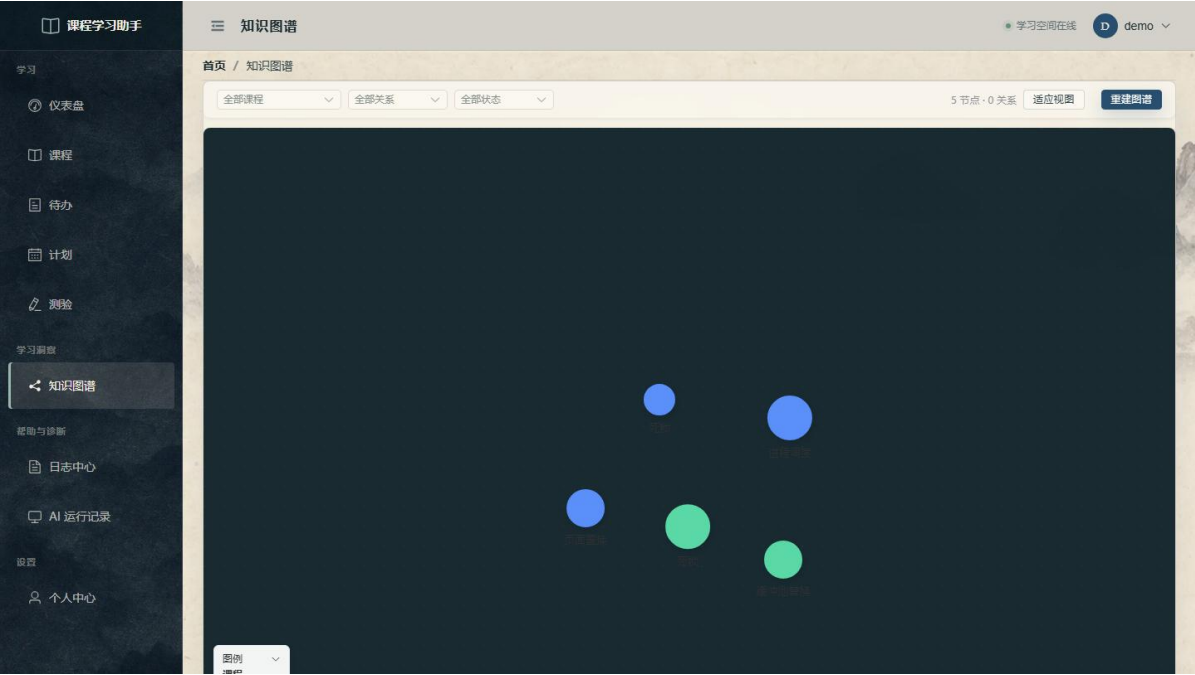


图 5-7 跨课程知识图谱

知识图谱采用深墨夜色画布承载节点网络，使用颜色和关系类型表示相似、对比、前置、易混和上下位等联系，支持课程、关系和状态筛选。

5.5 基于资料的安全问答



图 5-8 证据不足处理

当用户提出的概念无法从当前课程资料中得到可验证支撑时，系统返回“证据不

足”提示，并显示引用资料数量和检索过程，不输出未经证实的课程结论。

5.6 学习计划与待办执行

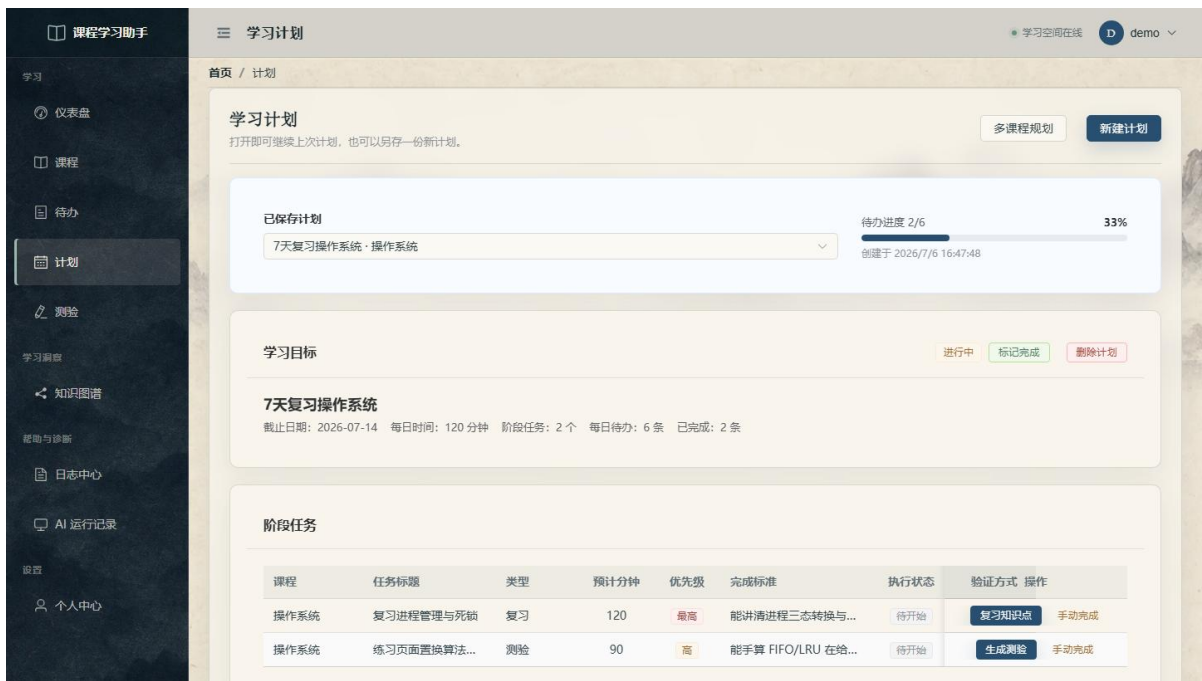


图 5-9 学习计划与阶段任务

学习计划根据截止日期和每日可用时间生成阶段任务。任务记录课程、类型、预计时长、优先级、完成标准、执行状态和验证方式，并可直接进入知识学习或生成测验。

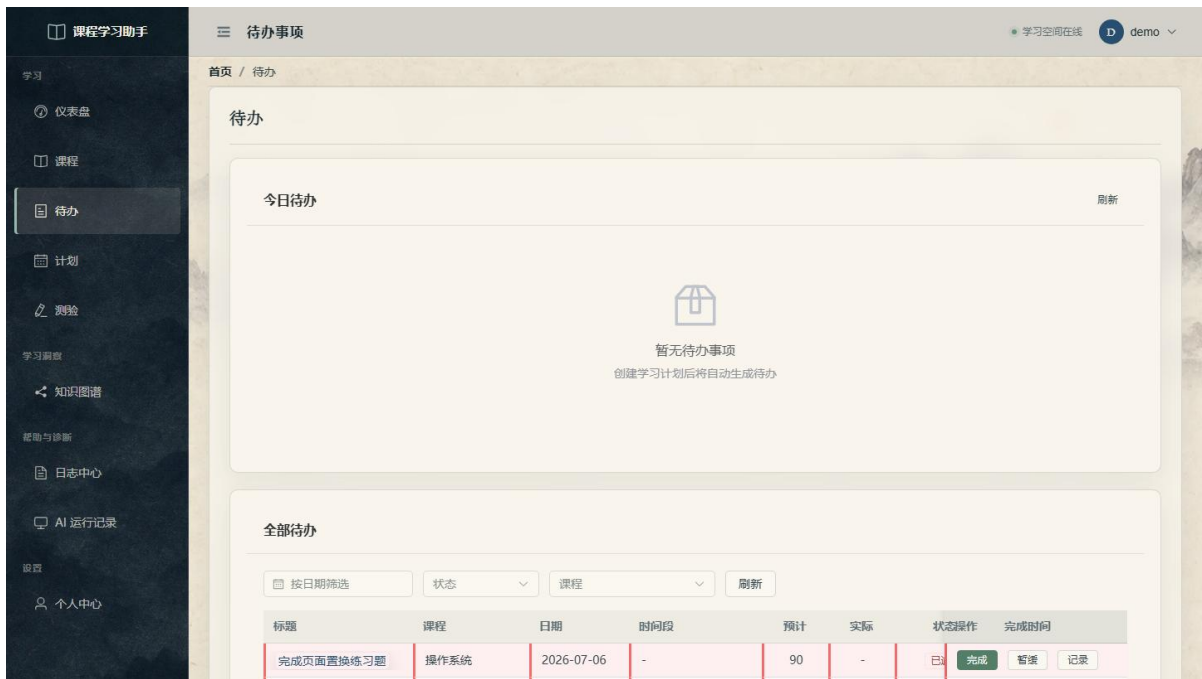


图 5-10 今日待办和历史记录

待办页面同时展示今日任务卡片和全部待办表格，支持完成、暂停、记录时长和撤销。历史记录保留计划日期、预计/实际用时、状态和完成时间。

5.7 模型配置和运行审计

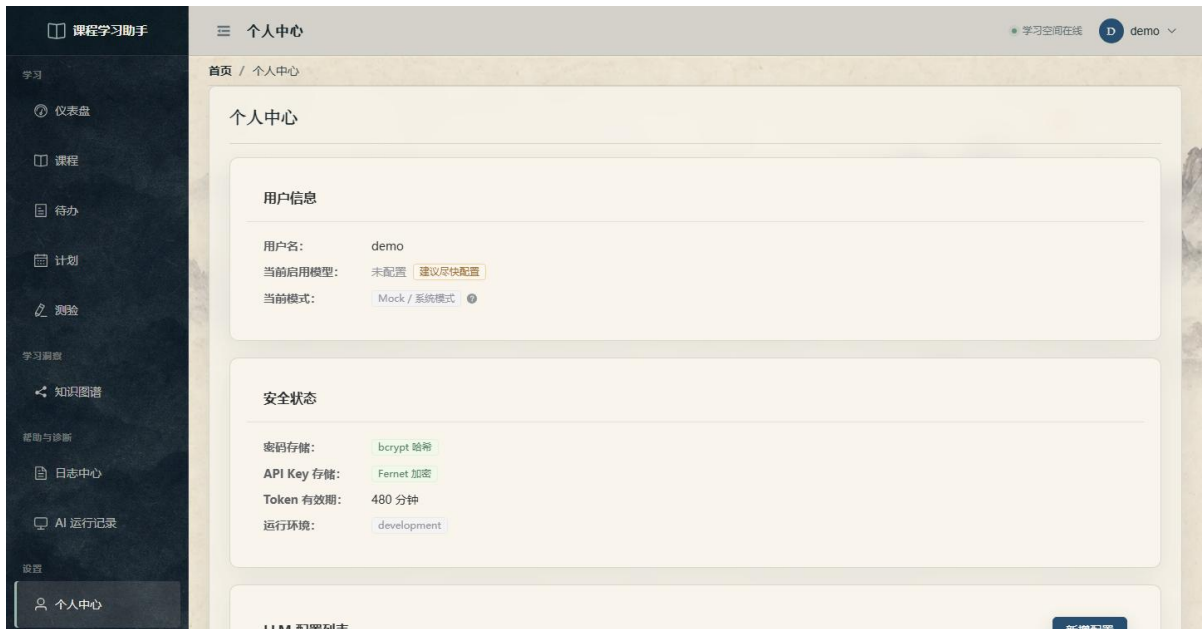


图 5-11 个人中心与真实模型配置

个人中心显示当前启用模型、用户配置模式、安全状态和配置列表。示例配置连接测试成功，API Key 只显示脱敏字符；页面同时显示密码哈希和密钥加密状态。

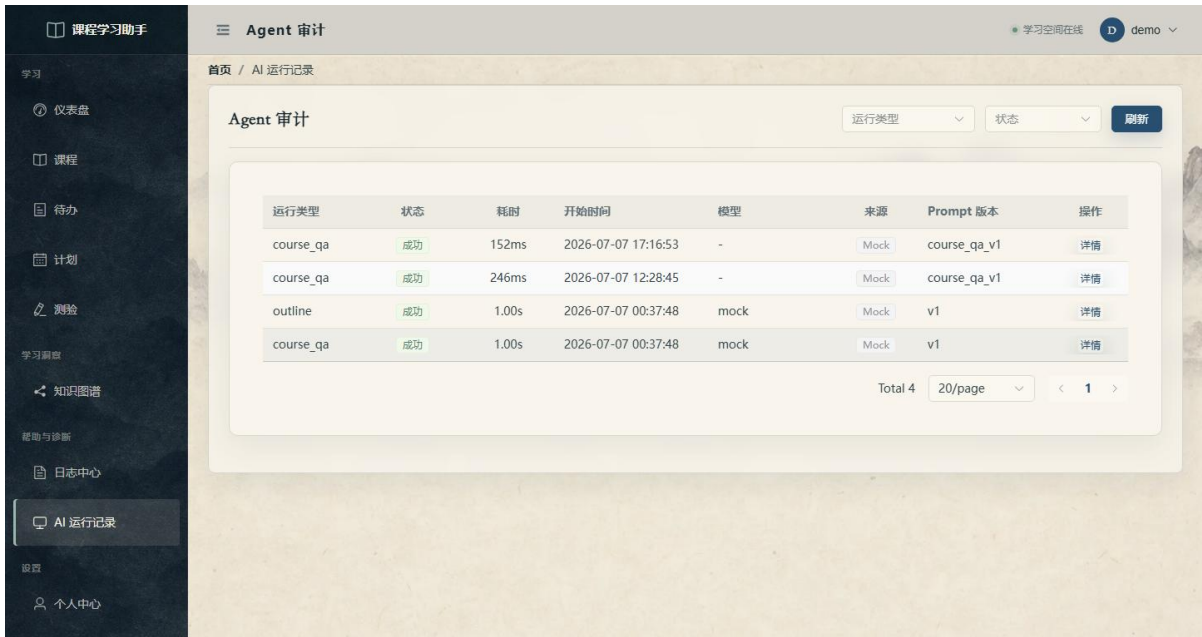


图 5-12 Agent 运行详情

运行详情记录开始/结束时间、耗时、Prompt 版本、请求与实际 Provider/Model、Fallback 状态、输入摘要、输出摘要和步骤明细。截图中的问答因没有可验证原文而被标记为证据不足，审计记录与用户界面保持一致。

5.8 持续集成与版本发布

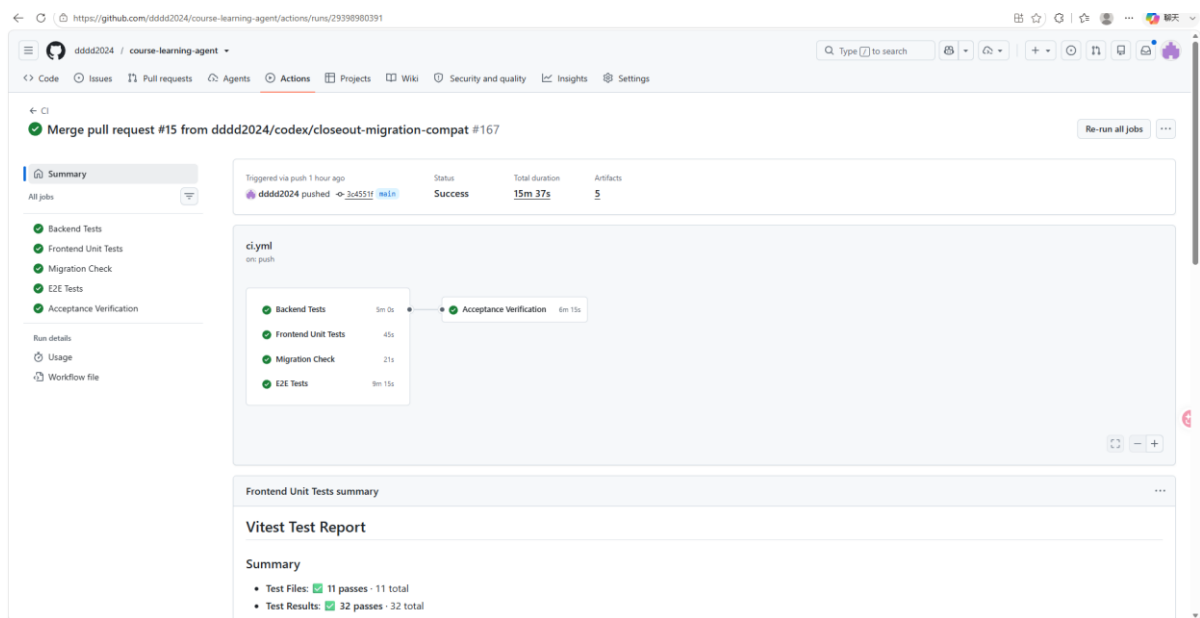


图 5-13 GitHub Actions 完整 CI

最新 main 提交的 CI 状态为 Success，总耗时约 15 分钟。Backend Tests、Frontend Unit Tests、Migration Check、E2E Tests 和 Acceptance Verification 五个 Job 均为绿色；前端报告显示 11 个测试文件、32 个测试用例全部通过。

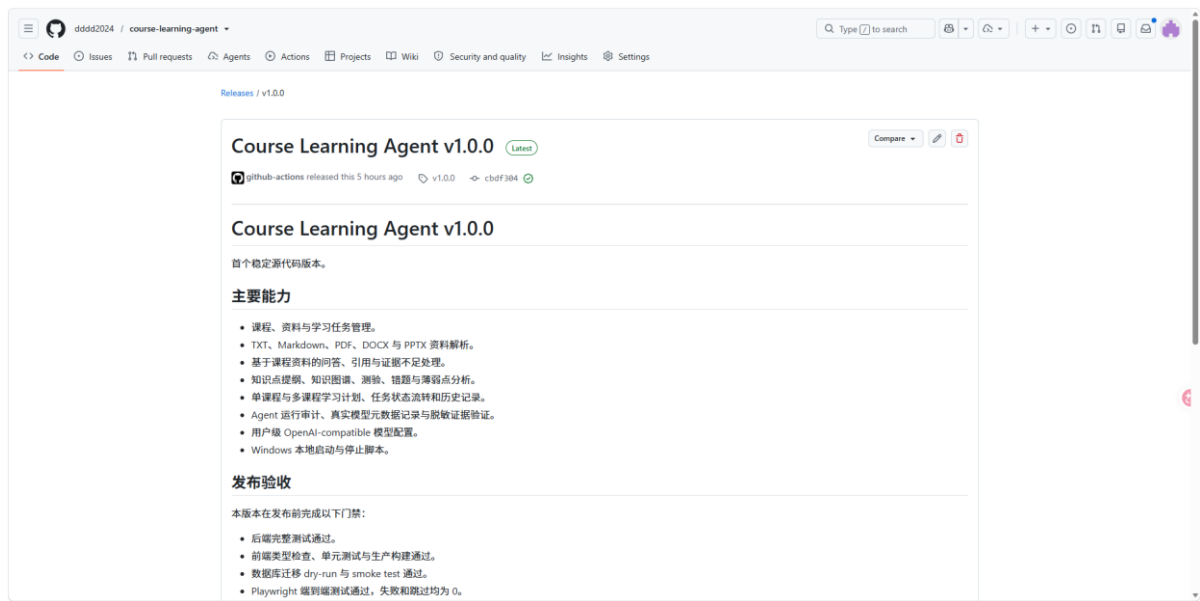


图 5-14 v1.0.0 Release 页面

项目已经发布 Course Learning Agent v1.0.0 稳定版本。Release 说明列出课程资料管理、问答与引用、知识点与图谱、计划与评测、Agent 审计、模型配置和 Windows 启动脚本，并声明后端、前端、迁移、E2E、V7 闭环和真实 LLM 验收均通过。

5.9 测试与验收结果

表 5-1 测试与验收结果

验收项目	方式	结果
后端测试	GitHub Actions: Backend Tests	完整测试通过。
前端类型检查	vue-tsc --noEmit	通过。
前端单元测试	Vitest	11 个测试文件、32 个测试用例通过。
前端生产构建	Vite build	通过。
数据库迁移	dry-run + smoke test	通过，迁移结果无错误。
端到端测试	Playwright Chromium	通过，失败和跳过均为 0。
功能闭环验收	Acceptance Verification / V7	全部通过。
真实模型验收	两轮，每轮 6 个场景	均完成 6/6，未出现 Mock、Fallback、Degraded 或元数据缺失。
仓库卫生	Repository Hygiene	未追踪运行时二进制、缓存、数据库、日志、报告、压缩包和凭据。

从功能展示和自动化验收结果可以看出，系统已经覆盖课程题目要求中的课程管理、资料管理、Agent 对话、学习计划、个人中心、来源引用、知识点整理、智能任务拆解和多课程规划，并进一步实现知识图谱、测验、薄弱点、模型配置、运行审计、日志诊断和自动化发布。

六、设计心得

本次实践最直接的体会是，大型程序设计的难点并不在于完成某一个页面或调用一次大模型，而在于让多个模块共享一致的数据边界和状态。资料解析、问答、知识点、计划和测验都依赖课程与资料版本；如果早期没有设计稳定的主键、版本和来源字段，后期很容易出现页面显示正常但引用、任务或测验无法追溯的问题。

第二个体会是，生成式 AI 功能必须设置可验证边界。仅把检索结果拼接进提示词并不能保证回答可靠，仍需检查引用原文、页码、主张支撑和实际调用模型。证据不足并不是系统失败，而是一种必要的安全输出。Agent 审计也使调试从“猜模型为什么这样答”转变为查看每一步输入、输出和回退链路。

第三个体会是，自动化测试能够显著降低复合系统的回归风险。项目中一次数据库字段或解析结构调整可能同时影响问答、计划、测验和前端页面。通过后端测试、前端类型检查、端到端测试、迁移检查和最终验收的分层门禁，可以在代码合并前定位问题。发布流程由 CI 驱动，也避免了把未经验证的本地状态直接标记为稳定版本。

第四个体会是，需要诚实描述系统边界。本版本使用 SQLite FTS5/BM25，而不是已经实现向量数据库；Mock 模式用于确定性流程演示，而不代表真实模型效果；本地单机版本也不等同于高并发生产系统。明确这些边界能够使报告、代码和答辩保持一致，并为后续扩展提供清晰起点。

通过本次项目，我进一步掌握了前后端分离开发、数据库建模、文档解析、检索增强生成、Agent 审计、测试驱动修复、Git 分支与 Pull Request、持续集成和版本发布等工程实践。相比单独完成若干算法题，本项目更强调需求、设计、实现、测试和交付之间的闭环。

6.1 后续改进方向

在保持引用验证的前提下，引入可选向量召回并与 FTS5 进行混合检索。

增加后台任务队列和解析进度推送，提升大文件与多文件并发处理体验。

根据薄弱点、任务延期和实际用时自动触发计划重排，并提供变更原因对比。

扩展知识图谱关系抽取和人工审核工具，支持更直观的课程间概念对比。

补充容器化部署、PostgreSQL 适配、对象存储和多实例运行方案。

七、团队分工说明

本项目由一人独立完成，不存在多人之间的任务拆分。本人负责从需求分析到版本发布的全部工作，包括系统规划、架构设计、数据库设计、后端 API 与业务服务、前端页面与交互、资料解析与检索问答、知识点和知识图谱、学习计划与待办、测验与薄弱点、模型配置与 Agent 审计、测试、数据库迁移、GitHub Actions、Release、演示数据和设计报告整理。

表 7-1 个人工作说明

工作类别	个人完成内容
需求与项目计划	分析课程题目要求，确定功能范围、实现边界和验收标准。
总体与数据库设计	完成前后端架构、领域模型、表结构、版本和追溯关系设计。
后端实现	完成 FastAPI 接口、服务层、ORM、资料解析、检索、Agent、计划和测验逻辑。
前端实现	完成 Vue 页面、组件、路由、状态管理、表单、图谱和审计展示。
安全与配置	完成 JWT、bcrypt、Fernet、用户隔离、脱敏和模型连接测试。
测试与修复	编写和执行 pytest、Vitest、Playwright、迁移与功能闭环验收，修复回归问题。
工程交付	维护 Git 分支与 PR，配置 CI、发布 v1.0.0，整理截图、演示数据和报告。

虽然开发过程中使用了 AI 编程工具辅助分析和生成部分草案，但需求决策、代码审查、测试执行、错误修复、结果验收和最终提交均由本人负责。

八、AI 伦理声明

本项目包含生成式人工智能功能，并在开发过程中使用 ChatGPT、Codex 等 AI 工具辅助需求梳理、代码草案、测试用例补全和文档表达。AI 工具仅作为辅助工具，不拥有项目决策权；所有架构选择、代码合并、测试结果、截图和报告内容均由本人检查确认。

在数据与隐私方面，未向外部模型提交真实密码、完整 API Key、个人隐私数据或未经授权的课程资料。用户级 API Key 在系统中加密保存，前端和截图只显示脱敏形式；本地数据库、上传资料、解析产物、日志和环境配置均排除在公开仓库之外。

在内容可靠性方面，系统不把大模型输出直接当作事实。课程问答先检索资料，再进行引用验证；回答无法得到资料支撑时显示“证据不足”。Agent 审计保存请求模型、实际模型、回退和步骤信息，使 Mock、真实模型和降级结果能够被识别。

在学术诚信方面，本报告用于说明本人完成的系统设计、实现和测试过程。AI 生成或辅助修改的内容经过本人理解、重写和核验，不以 AI 替代个人对代码和原理的掌握。项目中引用的开源框架和依赖遵循其许可证，未复制他人课程设计成品。

8.1 责任承诺

本人对所提交源代码、运行结果、测试记录和设计报告的真实性的负责，并能够在答辩中解释系统架构、数据库关系、检索问答流程、计划生成、Agent 审计、测试门禁及主要代码实现。